



MASTER D'EXCELLENCE INTELLIGENCE ARTIFICIELLE

TRAVAUX DIRIGÉS — MACHINE LEARNING

## TD N° 03

### Analyse Exploratoire des Données (EDA Avancée)

|                                            |                                                     |
|--------------------------------------------|-----------------------------------------------------|
| <b>Durée</b> : 2 heures                    | <b>Niveau</b> : Débutant – Intermédiaire            |
| <b>Dataset</b> : BankChurners (suite TD 2) | <b>Outils</b> : pandas, seaborn, matplotlib, plotly |

#### Objectifs du TD

- Produire une **EDA complète et structurée** sur un dataset bancaire réel
- Analyser la **distribution de la variable cible** et ses implications pour la modélisation
- Visualiser les **variables catégorielles** et leur relation avec le churn
- Explorer les **distributions numériques** : asymétrie, outliers, modalités
- Construire et interpréter une **matrice de corrélation** (Pearson et Spearman)
- Produire des **visualisations bivariées** avancées (KDE, violin, pairplot, scatter)
- Formuler des **hypothèses métier** à partir des données visuelles

Pr. BEN LAHMAR EL Habib · Semestre 2024–2025



## Contexte et Rappels Théoriques

Ce TD fait suite au TD 2 dans lequel vous avez prétraité le dataset BankChurners. Vous allez maintenant l'**explorer visuellement** pour comprendre ses structures, identifier les variables discriminantes et formuler des hypothèses avant la modélisation.

### ❓ Pourquoi l'EDA avant la modélisation ?

- **Comprendre les données** : types, distributions, plages de valeurs
- **Détecter les anomalies** : outliers, valeurs aberrantes, incohérences
- **Identifier les features discriminantes** : lesquelles séparent le mieux les classes ?
- **Guider le feature engineering** : quelles transformations sont utiles ?
- **Éviter les surprises** : un modèle entraîné sur des données mal comprises est imprévisible

## Rappels — Types de visualisations

| Graphique           | Données adaptées   | Librairie  | Usage principal                  |
|---------------------|--------------------|------------|----------------------------------|
| Histogramme         | Numérique continu  | sns/plt    | Distribution d'une variable      |
| KDE (densité)       | Numérique continu  | seaborn    | Distribution lissée, comparaison |
| Boxplot             | Numérique continu  | sns/plt    | Médiane, IQR, outliers           |
| Violin plot         | Numérique + catég. | seaborn    | Distribution + densité           |
| Countplot / Barplot | Catégorielle       | seaborn    | Fréquences par modalité          |
| Heatmap             | Matrice de corr.   | seaborn    | Corrélations toutes paires       |
| Scatterplot         | 2 numériques       | sns/plotly | Relation entre 2 variables       |
| Pairplot            | Multivariée        | seaborn    | Toutes paires de variables       |
| Sunburst / Treemap  | Hiérarchique       | plotly     | Exploration croisée interactive  |

### ❓ Rappel de cours — Corrélation de Pearson vs Spearman

- **Pearson (r)** : mesure la relation \*linéaire\* entre deux variables continues. Sensible aux outliers. Hypothèse de normalité.
- **Spearman (ρ)** : mesure la relation \*monotone\* (via les rangs). Robuste aux outliers. Capture les relations non-linéaires.
- **Règle pratique** : si les variables ont des outliers ou des distributions asymétriques → préférer Spearman.



- **Seuils d'interprétation** :  $|r| < 0,3$  = faible /  $0,3-0,7$  = modéré /  $> 0,7$  = fort

**Formule Pearson** :  $r = \text{Cov}(X, Y) / (\sigma_X \times \sigma_Y)$       **Spearman** :  $\rho = 1 - 6\sum d_i^2 / n(n^2-1)$

#### 🔍 Rappel de cours — Asymétrie (Skewness) et Kurtosis

- **Asymétrie positive (skew > 0)** : queue à droite, la majorité des valeurs est à gauche. Ex : revenus, transactions.
- **Asymétrie négative (skew < 0)** : queue à gauche. Ex : âge de retraite.
- **Kurtosis > 3** (leptokurtique) : distribution à queues épaisses, plus d'outliers que la normale.
- **Kurtosis = 3** (normale) / **< 3** (platykurtique) : queues fines.
- **Implication ML** : une forte asymétrie peut nécessiter une transformation logarithmique avant la normalisation.

Avant de commencer les exercices, chargez les données prétraitées depuis le TD 2 :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Charger les données brutes (ou depuis le preprocessing TD2)
raw = pd.read_csv('BankChurners.csv')
data = raw[raw.columns[:-2]].copy()

# Configuration globale des graphiques
sns.set_style('whitegrid')
sns.set_palette('Set2')
matplotlib.rcParams['figure.dpi'] = 110
matplotlib.rcParams['figure.figsize'] = (12, 6)
pd.set_option('display.max_columns', 30)

print(f'Dataset chargé : {data.shape[0]} lignes × {data.shape[1]} colonnes')
```



## Partie 1 — Variable Cible et Déséquilibre des Classes

**Durée estimée :** 20 minutes | La toute première question d'une EDA : **ma variable cible est-elle équilibrée ?**

### Q 1.1

#### Distribution de la variable cible — Attrition\_Flag

##### Consigne

Visualisez la distribution de la variable cible avec plusieurs représentations complémentaires. Calculez les métriques d'équilibre et discutez des implications pour la modélisation.

```
# Distribution brute
target_counts = data['Attrition_Flag'].value_counts()
target_pct    = data['Attrition_Flag'].value_counts(normalize=True) * 100

print('=== DISTRIBUTION DE LA VARIABLE CIBLE ===')
for label in target_counts.index:
    print(f'  {label:25s} : {target_counts[label]:5d} ({target_pct[label]:.1f}%)')

# Visualisation - Barplot + Pie chart
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# 1. Barplot annoté
bars = axes[0].bar(target_counts.index, target_counts.values,
                  color=['#1A7A6E', '#9C3D38'], edgecolor='black', linewidth=.8)
for bar, pct in zip(bars, target_pct.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
                f'{pct:.1f}%', ha='center', fontweight='bold', fontsize=12)
axes[0].set_title('Distribution de la Variable Cible', fontweight='bold',
                fontsize=13)
axes[0].set_ylabel('Nombre de clients')

# 2. Pie chart
axes[1].pie(target_counts.values, labels=target_counts.index,
            autopct='%1.1f%%', colors=['#1A7A6E', '#9C3D38'],
            startangle=90, wedgeprops={'edgecolor':'white', 'linewidth':2})
axes[1].set_title('Proportion Existing vs Attrited', fontweight='bold', fontsize=13)

plt.suptitle('Analyse de la Variable Cible - BankChurners',
            fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()
```

#### 🔗 Déséquilibre de classes — Impact critique sur la modélisation



Avec **~16,1 % de churns** seulement, un classifieur naïf qui prédirait toujours 'Existing Customer' obtiendrait **83,9 % d'accuracy** sans avoir rien appris ! Ce TD prépare visuellement la solution apportée en TD 5 (SMOTE).

## Questions d'analyse

1. Quel est le ratio exact Existing / Attrited ? Qualifiez ce déséquilibre en vous référant au tableau du cours (mild / moderate / extreme).
2. Pourquoi l'**accuracy** est-elle une métrique trompeuse sur ce dataset ? Quelle(s) métrique(s) faudra-t-il privilégier lors de l'évaluation des modèles ?
3. Si vous entraînez un modèle sans corriger ce déséquilibre, vers quelle classe va-t-il avoir tendance à sur-prédire ? Pourquoi ?

### Q 1.2

#### Ratio de churn par segment — première exploration croisée

##### Consigne

Calculez le taux de churn (% de clients Attrited) par grande catégorie. Identifiez les profils de clients les plus à risque.

```
# Encoder la cible pour les calculs
data['churn'] = (data['Attrition_Flag'] == 'Attrited Customer').astype(int)

# Taux de churn par variable catégorielle
cat_vars = ['Gender', 'Education_Level', 'Income_Category',
            'Marital_Status', 'Card_Category']

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for ax, col in zip(axes[:5], cat_vars):
    churn_rate = data.groupby(col)['churn'].mean().sort_values(ascending=False) * 100
    bars = ax.bar(churn_rate.index, churn_rate.values,
                  color='#1A2550', alpha=.8, edgecolor='white')
    # Ligne de référence : taux global
    ax.axhline(data['churn'].mean()*100, color='#9C3D38',
               ls='--', lw=2, label=f'Moy. globale\n({data["churn"].mean()*100:.1f}%)')
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x()+bar.get_width()/2, h+.3, f'{h:.1f}%',
               ha='center', fontsize=9, fontweight='bold')
    ax.set_title(f'Churn rate par {col}', fontweight='bold')
```



```
ax.set_ylabel('Churn rate (%)')
ax.tick_params(axis='x', rotation=25)
ax.legend(fontsize=8)

fig.delaxes(axes[-1])
plt.suptitle('Taux de Churn par Segment Catégoriel',
            fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()
```

### Questions d'analyse

4. Quelle variable catégorielle présente la plus grande variation du taux de churn entre ses modalités ? Quel est l'écart entre la modalité la plus risquée et la moins risquée ?
5. Pour **Card\_Category** : y a-t-il une corrélation entre le type de carte et le risque de churn ? Formulez une hypothèse métier explicative.
6. La variable **Gender** est-elle discriminante pour le churn ? Justifiez avec les chiffres.



## Partie 2 — Analyse des Variables Catégorielles

**Durée estimée :** 25 minutes | Explorer la distribution interne des catégories et leur relation avec la variable cible.

### Q 2.1

### Distribution des 5 variables catégorielles

#### Consigne

Produisez un tableau de bord visuel des 5 variables catégorielles avec les fréquences annotées et la décomposition par Attrition\_Flag.

```
categoricals = ['Gender', 'Education_Level', 'Marital_Status',
               'Income_Category', 'Card_Category']

fig, axes = plt.subplots(2, 3, figsize=(20, 12))
axes = axes.flatten()

for idx, col in enumerate(categoricals):
    ax = axes[idx]
    # Countplot avec hue = Attrition_Flag
    order = data[col].value_counts().index
    sns.countplot(x=col, hue='Attrition_Flag', data=data,
                  ax=ax, order=order,
                  palette=['#1A7A6E', '#9C3D38'])
    ax.legend(title='Statut', labels=['Existing', 'Attrited'],
              fontsize=8, title_fontsize=8)
    ax.set_title(f'Distribution de {col}', fontweight='bold', fontsize=11)
    ax.set_xlabel('')
    if col in ['Education_Level', 'Income_Category']:
        ax.tick_params(axis='x', rotation=25)
    # Annoter les barres avec les %
    total = len(data)
    for p in ax.patches:
        h = p.get_height()
        if h > 0:
            ax.text(p.get_x()+p.get_width()/2., h+15,
                    f'{h/total*100:.1f}%', ha='center', fontsize=7.5)

fig.delaxes(axes[-1])
plt.suptitle('Distribution des Variables Catégorielles – BankChurners',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout() ; plt.show()
```

## Questions d'analyse



7. Quelle est la modalité majoritaire pour chacune des 5 variables ? Ces majorités vous semblent-elles cohérentes avec le profil d'un client bancaire américain ?
8. Pour **Education\_Level** : est-ce que le niveau d'éducation semble lié au churn ? Observez-vous une tendance monotone ?
9. Pour **Card\_Category** : la carte **Blue** est dominante à plus de 93%. Qu'est-ce que cela implique pour l'interprétation des résultats de churn par type de carte ?

## Q 2.2

### Visualisation interactive Sunburst — exploration croisée

#### Consigne

Créez un graphique Sunburst Plotly pour explorer simultanément 3 niveaux de hiérarchie : **Attrition\_Flag** → **Gender** → **Card\_Category**. Analysez les sous-populations spécifiques.

```
fig = px.sunburst(
    data,
    path=['Attrition_Flag', 'Gender', 'Card_Category'],
    values='Credit_Limit',
    title='Exploration Hiérarchique : Attrition → Genre → Type de Carte',
    color_discrete_sequence=px.colors.qualitative.Set2
)
fig.update_layout(height=580, title_font_size=14)
fig.show()

# Variante : croiser Income_Category et Education_Level
fig2 = px.sunburst(
    data,
    path=['Attrition_Flag', 'Income_Category', 'Education_Level'],
    title='Attrition → Revenu → Éducation',
    color_discrete_sequence=px.colors.qualitative.Pastel
)
fig2.update_layout(height=580)
fig2.show()
```

#### 🔗 Comment lire un graphique Sunburst ?

- Le **centre** représente la variable racine (Attrition\_Flag)
- Chaque **anneau** ajoute un niveau de granularité
- La **surface de chaque segment** est proportionnelle à la valeur agrégée (ici Credit\_Limit)
- **Cliquer** sur un segment zoome dans la sous-hiérarchie correspondante

## Questions d'analyse





- 
10. Dans le Sunburst Attrition → Gender → Card\_Category : quelle sous-population (genre × type de carte) a le taux de churn apparent le plus élevé ?
11. Dans le second Sunburst (Attrition → Income → Éducation) : les clients à revenu < 40K et niveau 'Graduate' sont-ils sur- ou sous-représentés parmi les Attrited ?



## Partie 3 — Distributions des Variables Numériques

**Durée estimée :** 30 minutes | Comprendre la forme, l'étendue et les particularités de chaque variable numérique.

### Q 3.1

#### Statistiques descriptives avancées — Asymétrie et Kurtosis

##### Consigne

Calculez les statistiques descriptives complètes incluant l'asymétrie (skewness) et le kurtosis. Classez les variables selon leur degré d'asymétrie.

```
from scipy import stats

num_cols = ['Customer_Age', 'Credit_Limit', 'Months_on_book',
            'Total_Relationship_Count', 'Months_Inactive_12_mon',
            'Contacts_Count_12_mon', 'Total_Revolving_Bal',
            'Avg_Open_To_Buy', 'Total_Amt_Chng_Q4_Q1',
            'Total_Trans_Amt', 'Total_Trans_Ct',
            'Total_Ct_Chng_Q4_Q1', 'Avg_Utilization_Ratio']

# Tableau complet
desc = data[num_cols].describe().T
desc['skewness'] = data[num_cols].skew()
desc['kurtosis'] = data[num_cols].kurtosis()
desc['cv'] = desc['std'] / desc['mean'] # Coefficient de variation

print('=== STATISTIQUES DESCRIPTIVES COMPLÈTES ===')
print(desc[['mean', 'std', 'min', '25%', '50%', '75%', 'max',
            'skewness', 'kurtosis']].round(3).to_string())

# Classer par asymétrie décroissante
print('\n=== CLASSEMENT PAR ASYMÉTRIE ===')
print(desc['skewness'].sort_values(ascending=False).round(3))
```

##### 📌 Rappel de cours — Interpréter l'asymétrie (skewness)

- $|\text{skew}| < 0,5$  : distribution quasi-symétrique → StandardScaler acceptable
- $0,5 \leq |\text{skew}| < 1$  : asymétrie modérée → RobustScaler recommandé
- $|\text{skew}| \geq 1$  : forte asymétrie → envisager transformation  $\log(x+1)$  ou  $\sqrt{x}$
- **Attention** : le kurtosis mesure les queues de distribution, pas l'asymétrie. Une distribution peut être symétrique mais à queues très épaisses (fort kurtosis).



## Questions d'analyse

12. Identifiez les 3 variables ayant l'asymétrie la plus forte. Pour chacune, proposez une transformation adaptée.
13. La variable **Customer\_Age** est-elle asymétrique ? Quelle distribution semble-t-elle suivre visuellement ?
14. Calculez le ratio moyenne/médiane pour **Credit\_Limit**. Que vous dit un ratio  $> 1$  sur la distribution ?

### Q 3.2

### Distributions comparées — KDE Existing vs Attrited

#### Consigne

Produisez des graphiques KDE superposant les distributions des clients Existing et Attrited pour les 6 variables numériques clés. Identifiez visuellement les features les plus discriminantes.

```
key_num = ['Credit_Limit', 'Total_Trans_Amt', 'Total_Trans_Ct',
            'Total_Revolving_Bal', 'Avg_Utilization_Ratio',
            'Months_Inactive_12_mon']

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for ax, col in zip(axes, key_num):
    # Existing
    sns.kdeplot(
        data.loc[data['Attrition_Flag']=='Existing Customer', col],
        ax=ax, label='Existing', color='#1A7A6E', fill=True, alpha=.35)
    # Attrited
    sns.kdeplot(
        data.loc[data['Attrition_Flag']=='Attrited Customer', col],
        ax=ax, label='Attrited', color='#9C3D38', fill=True, alpha=.35)
    # Médianes
    for flag, color in [('Existing Customer', '#1A7A6E'),
                        ('Attrited Customer', '#9C3D38')]:
        med = data.loc[data['Attrition_Flag']==flag, col].median()
        ax.axvline(med, color=color, ls='--', lw=1.8,
                    label=f'Médiane {flag[:8]}={med:.0f}')
    ax.set_title(col, fontweight='bold', fontsize=11)
    ax.legend(fontsize=7)

plt.suptitle('Distributions KDE — Existing vs Attrited',
             fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()
```



#### Comment lire les graphiques KDE comparatifs ?

- Si les deux courbes sont **très séparées** → la variable est **discriminante** (utile pour le modèle)
- Si les deux courbes se **superposent presque totalement** → la variable est **peu informative**
- Regardez les **pics (modes)** : une variable avec un pic à 0 uniquement pour les Attrited est un signal très fort
- Les **médianes en pointillés** permettent de comparer les centres de distribution

### Questions d'analyse

15. Pour **Total\_Revolving\_Bal** : observez-vous un pic particulier pour les clients Attrited ? Quelle interprétation métier proposez-vous ?
16. Pour **Avg\_Utilization\_Ratio** : la distribution des Attrited présente-t-elle une particularité autour de 0 ? Qu'est-ce que cela signifie ?
17. Classez les 6 variables par pouvoir discriminant (de la plus à la moins discriminante) en vous basant sur la séparation visuelle des courbes KDE.
18. Pour **Months\_Inactive\_12\_mon** : quel seuil d'inactivité (nombre de mois) semble le plus associé au churn ?

#### Q 3.3

#### Boxplots et Violin Plots — Outliers et variabilité

##### Consigne

Combinez boxplots et violin plots pour visualiser à la fois les outliers, les quartiles ET la densité de distribution par groupe.

```
fig, axes = plt.subplots(2, 2, figsize=(16, 11))

# Violin + boxplot superposés
for ax, col in zip(axes.flatten(),
                  ['Total_Trans_Amt', 'Credit_Limit',
                  'Total_Revolving_Bal', 'Avg_Utilization_Ratio']):
    sns.violinplot(x='Attrition_Flag', y=col, data=data, ax=ax,
                  palette=['#1A7A6E', '#9C3D38'],
                  inner='box', # boxplot interne
                  cut=0) # limites réelles des données
    ax.set_title(f'Violin - {col}', fontweight='bold', fontsize=11)
    ax.set_xlabel('')
    ax.tick_params(axis='x', rotation=15)

plt.suptitle('Violin Plots - Distribution par Statut Client',
            fontsize=14, fontweight='bold')
```



```
plt.tight_layout() ; plt.show()

# Distribution de l'âge par tranche (pd.cut)
data['Age_Group'] = pd.cut(data['Customer_Age'], bins=5)
fig2, ax2 = plt.subplots(figsize=(12, 5))
sns.countplot(y='Card_Category', hue='Age_Group', data=data, ax=ax2)
ax2.set_title('Type de carte par tranche d\'âge', fontweight='bold')
plt.tight_layout() ; plt.show()
```

## Questions d'analyse

19. Le violin plot de **Credit\_Limit** : la distribution est-elle similaire entre Existing et Attrited ? La variabilité (largeur du violin) est-elle comparable ?
20. Pour **Avg\_Utilization\_Ratio** : que représente la bosse à 0 dans le violin des Attrited ? Est-ce un outlier ou un groupe cohérent ?
21. La tranche d'âge influence-t-elle le type de carte souscrit ? Observez-vous des tendances par génération ?



## Partie 4 — Corrélations et Relations Bivariées

**Durée estimée :** 25 minutes | Identifier les features corrélées entre elles et avec la cible.

### Q 4.1

#### Matrice de corrélation — Pearson et Spearman

##### Consigne

Calculez et comparez les matrices de corrélation de Pearson et Spearman. Identifiez les paires fortement corrélées ( $|r| > 0,7$ ) et discutez des implications.

```
num_for_corr = ['Customer_Age', 'Credit_Limit', 'Months_on_book',
                'Total_Relationship_Count', 'Months_Inactive_12_mon',
                'Contacts_Count_12_mon', 'Total_Revolving_Bal',
                'Avg_Open_To_Buy', 'Total_Trans_Amt',
                'Total_Trans_Ct', 'Avg_Utilization_Ratio']

corr_pearson = data[num_for_corr].corr('pearson')
corr_spearman = data[num_for_corr].corr('spearman')

fig, axes = plt.subplots(1, 2, figsize=(20, 9))

for ax, corr, title in zip(
    axes,
    [corr_pearson, corr_spearman],
    ['Pearson (linéaire)', 'Spearman (monotone)']
):
    mask = np.triu(np.ones_like(corr, dtype=bool))
    sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
                cmap='RdYlGn', center=0, vmin=-1, vmax=1,
                square=True, linewidths=.5,
                annot_kws={'size':8}, ax=ax)
    ax.set_title(f'Corrélation {title}', fontweight='bold', fontsize=12)
    ax.tick_params(axis='x', rotation=40)
    ax.tick_params(axis='y', rotation=0)

plt.suptitle('Matrices de Corrélation — BankChurners',
             fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()

# Fortes corrélations
print('=== FORTES CORRÉLATIONS (|r| > 0.7 — Pearson) ===')
for i in range(len(corr_pearson.columns)):
    for j in range(i+1, len(corr_pearson.columns)):
        r = corr_pearson.iloc[i, j]
        if abs(r) > 0.7:
```



```
print(f' {corr_pearson.columns[i]:30s} ↔ '  
      f'{corr_pearson.columns[j]:30s} : {r:.3f}')
```

## Questions d'analyse

22. Quelle paire de variables présente la corrélation de Pearson la plus élevée ? Expliquez pourquoi cette corrélation est attendue d'un point de vue métier.
23. Comparez les valeurs Pearson et Spearman pour `Total_Trans_Amt` ↔ `Total_Trans_Ct`. Sont-elles proches ? Qu'est-ce que cela indique sur la nature de la relation ?
24. Pour la Régression Logistique, une forte multicollinéarité entre features pose problème. Quelles paires identifiées faudrait-il traiter ? Comment ? (Cf. TD 10 — Régularisation)
25. La matrice de corrélation capture-t-elle les relations non-linéaires ? Donnez un exemple de relation non-linéaire que Pearson ne peut pas détecter sur ce dataset.

### Q 4.2

### Corrélation des features avec la variable cible

#### Consigne

Calculez la corrélation de chaque feature numérique avec la variable cible churn (0/1). Classez-les du plus au moins informatif.

```
# Corrélation avec la cible (Spearman recommandé - données asymétriques)
churn_corr = data[num_for_corr + ['churn']].corr('spearman')['churn']
churn_corr = churn_corr.drop('churn').sort_values(key=abs, ascending=False)

# Barplot horizontal
fig, ax = plt.subplots(figsize=(10, 7))
colors = ['#9C3D38' if v < 0 else '#1A7A6E' for v in churn_corr.values]
bars = ax.barh(churn_corr.index[::-1], churn_corr.values[::-1],
               color=colors[::-1], edgecolor='white', linewidth=.8)
ax.axvline(0, color='black', linewidth=1)
ax.set_xlabel('Corrélation Spearman avec Churn', fontsize=12)
ax.set_title('Importance des Features pour Prédire le Churn',
             fontweight='bold', fontsize=13)

for bar, val in zip(bars, churn_corr.values[::-1]):
    ax.text(val + 0.005 * (1 if val >= 0 else -1),
            bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=9)
plt.tight_layout() ; plt.show()

print('=== TOP 5 FEATURES POUR LE CHURN (Spearman) ===')
print(churn_corr.head())
```



```
print('\n=== TOP 5 FEATURES LES MOINS INFORMATIVES ===')  
print(churn_corr.tail())
```

### Questions d'analyse

26. Quelles sont les 3 features les plus corrélées avec le churn ? Le signe (positif ou négatif) de la corrélation est-il cohérent avec ce que vous observiez dans les KDE de la Partie 3 ?
27. La variable `Customer_Age` est-elle un bon prédicteur du churn ? Comment l'expliquez-vous ?
28. La corrélation Spearman est-elle toujours supérieure à la corrélation Pearson pour ces features ? Pourquoi ?





## Partie 5 — Visualisations Avancées et Synthèse Métier

**Durée estimée :** 20 minutes | Croiser plusieurs variables pour dégager des patterns complexes et formuler des insights actionnables.

### Q 5.1

#### Scatter plot — Transactions vs Solde renouvelable

```
# Scatter bivarié coloré par Attrition
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

for ax, (x_col, y_col) in zip(axes, [
    ('Total_Trans_Amt', 'Total_Revolving_Bal'),
    ('Total_Trans_Ct', 'Avg_Utilization_Ratio')
]):
    for flag, color, alpha in [
        ('Existing Customer', '#1A7A6E', .4),
        ('Attrited Customer', '#9C3D38', .7)
    ]:
        subset = data[data['Attrition_Flag'] == flag]
        ax.scatter(subset[x_col], subset[y_col],
                   c=color, alpha=alpha, s=15, label=flag)
    ax.set_xlabel(x_col, fontsize=11)
    ax.set_ylabel(y_col, fontsize=11)
    ax.set_title(f'{x_col}\nvs {y_col}', fontweight='bold')
    ax.legend(fontsize=9)

plt.suptitle('Relations Bivariées — Existing vs Attrited',
             fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()
```

### Questions d'analyse

29. Dans le scatter **Total\_Trans\_Amt** vs **Total\_Revolving\_Bal** : observez-vous un groupe de clients Attrited avec un solde renouvelable exactement à 0 ? Quelle interprétation proposez-vous ?
30. Dans le scatter **Total\_Trans\_Ct** vs **Avg\_Utilization\_Ratio** : à partir de quel seuil approximatif du nombre de transactions les clients semblent-ils moins susceptibles de churner ?

### Q 5.2

#### Profil moyen — Comparaison Existing vs Attrited

##### Consigne

Construisez un tableau comparatif des moyennes des features numériques clés entre les deux groupes. Identifiez les écarts les plus significatifs.



```
att = data[data['Attrition_Flag'] == 'Attrited Customer']
exist = data[data['Attrition_Flag'] == 'Existing Customer']

key_metrics = ['Customer_Age', 'Months_on_book',
               'Total_Relationship_Count', 'Months_Inactive_12_mon',
               'Contacts_Count_12_mon', 'Credit_Limit',
               'Total_Revolving_Bal', 'Total_Trans_Amt',
               'Total_Trans_Ct', 'Avg_Utilization_Ratio']

# Tableau comparatif
comparison = pd.DataFrame({
    'Existing (moy.)': exist[key_metrics].mean(),
    'Attrited (moy.)': att[key_metrics].mean(),
})
comparison['Différence (%)'] = (
    (comparison['Attrited (moy.)'] - comparison['Existing (moy.)'])
    / comparison['Existing (moy.)'] * 100
).round(1)
comparison = comparison.round(2)

print('=== PROFIL MOYEN - EXISTING vs ATTRITED ===')
print(comparison.to_string())

# Radar chart (spider chart) - comparaison normalisée
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
norm = pd.DataFrame(
    scaler.fit_transform(comparison[['Existing (moy.)', 'Attrited (moy.)']]),
    index=comparison.index,
    columns=['Existing', 'Attrited']
)
print('\nProfil normalisé [0-1] :')
print(norm.round(3).to_string())
```

## Questions d'analyse

31. Quelle métrique présente la plus grande différence (en %) entre Existing et Attrited ? Formulez une règle métier simple basée sur ce constat.
32. Est-ce que les clients Attrited ont été dans la banque plus ou moins longtemps (**Months\_on\_book**) que les clients Existing ? Ce résultat vous surprend-il ?
33. À partir du tableau, formulez **3 hypothèses métier actionnables** — c'est-à-dire 3 conclusions que le directeur de la banque pourrait utiliser pour définir une stratégie de rétention.

### Q 5.3

### Synthèse — Les 5 insights clés de l'EDA



### Consigne

Dans votre notebook, rédigez une cellule Markdown de synthèse contenant les 5 observations les plus importantes de votre EDA, avec pour chacune : l'observation factuelle, l'interprétation métier et la recommandation pour la modélisation.

```
# Template de cellule Markdown à compléter :
# =====
# ## Synthèse EDA – 5 Insights Clés
#
# ### Insight 1 : [TITRE]
# **Observation** : ...
# **Interprétation métier** : ...
# **Implication pour le modèle** : ...
#
# ### Insight 2 : [TITRE]
# ...
# =====

# Pistes pour guider vos insights :
# - Taux de churn global et déséquilibre des classes
# - Variables numériques les plus discriminantes (KDE séparés)
# - Patterns remarquables : solde renouvelable = 0 chez Attrited
# - Corrélations fortes entre features → risque multicollinéarité
# - Variables peu informatives à considérer pour exclusion
```

### 🔗 Un bon insight EDA répond à 3 questions

- **Quoi ?** : Le fait observé dans les données (chiffre, tendance visuelle)
- **Pourquoi ?** : L'interprétation dans le contexte bancaire (comportement client)
- **Et alors ?** : Ce que cela implique pour choisir un modèle, des features, ou une métrique



## Section Bonus — Pour aller plus loin

Ces exercices approfondissent des techniques EDA moins courantes mais très utiles dans un contexte professionnel.

### Bonus B1 — Distribution bivariable KDE 2D

Visualisez la densité jointe de deux variables pour les deux classes de clients.

```
# KDE 2D - densité jointe
fig, axes = plt.subplots(1, 2, figsize=(16, 7))

for ax, flag in zip(axes, ['Existing Customer', 'Attrited Customer']):
    subset = data[data['Attrition_Flag'] == flag]
    sns.kdeplot(
        data=subset,
        x='Total_Trans_Amt',
        y='Total_Trans_Ct',
        ax=ax,
        fill=True,
        cmap='Blues' if flag=='Existing Customer' else 'Reds',
        levels=8
    )
    ax.set_title(f'KDE 2D - {flag}', fontweight='bold')

plt.suptitle('Densité jointe Trans_Amt × Trans_Ct',
             fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```

### Bonus B2 — PairPlot sélectif sur les top features

```
top_features = ['Total_Trans_Ct', 'Total_Revolving_Bal',
               'Avg_Utilization_Ratio', 'Contacts_Count_12_mon',
               'Attrition_Flag']

g = sns.pairplot(
    data[top_features],
    hue='Attrition_Flag',
    palette=['#1A7A6E', '#9C3D38'],
    diag_kind='kde',
    plot_kws={'alpha':0.4, 's':15},
    diag_kws={'alpha':0.5}
)

g.fig.suptitle('PairPlot - Top 4 Features vs Churn',
               y=1.02, fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```



34. Dans le PairPlot, identifiez la paire de features qui sépare le mieux les deux classes visuellement. Cette paire est-elle cohérente avec les corrélations calculées en Partie 4 ?
35. La diagonale montre les distributions individuelles de chaque feature par classe. Quelle feature présente la meilleure séparation sur la diagonale ?

### Bonus B3 — Analyse de Pareto du churn

La règle 80/20 s'applique souvent au churn : 20% des profils génèrent 80% des résiliations.

```
# Identifier les profils combinés à fort taux de churn
profile = data.groupby(['Income_Category', 'Education_Level'])
profile_churn = profile['churn'].agg(['mean', 'sum', 'count'])
profile_churn.columns = ['Taux_churn', 'N_attrited', 'N_total']
profile_churn = profile_churn.sort_values('Taux_churn', ascending=False)
profile_churn['Taux_churn'] = (profile_churn['Taux_churn']*100).round(1)

print('=== PROFILS Income x Education PAR TAUX DE CHURN ===')
print(profile_churn.head(10).to_string())
```



## Livraison attendue

### ✓ Ce que vous devez remettre

- Notebook **TD3\_NomPrenom.ipynb** — toutes les cellules exécutées avec outputs visibles
- Au minimum **10 visualisations** lisibles (titres, légendes, labels d'axes obligatoires)
- **Chaque visualisation commentée** : que vous apprend-elle ? Quelles décisions guide-t-elle ?
- Une cellule Markdown de **synthèse EDA** avec 5 insights structurés (Quoi / Pourquoi / Et alors ?)
- Délai : **avant le TD 4**, dépôt sur la plateforme pédagogique

## Critères d'évaluation

| Critère                                        | Coefficient | Description                  |
|------------------------------------------------|-------------|------------------------------|
| Qualité et richesse des visualisations         | 30%         | Variété, clarté, annotations |
| Pertinence des commentaires et interprétations | 35%         | Lien données ↔ métier        |
| Synthèse des 5 insights structurés             | 25%         | Quoi / Pourquoi / Et alors ? |
| Section Bonus (optionnel)                      | +10%        | Pairplot, KDE 2D ou Pareto   |

## Ressources et Documentation

- **Seaborn Gallery** : <https://seaborn.pydata.org/examples/>
- **Plotly Express** : <https://plotly.com/python/plotly-express/>
- **pandas — visualisation** : [https://pandas.pydata.org/docs/user\\_guide/visualization.html](https://pandas.pydata.org/docs/user_guide/visualization.html)
- **scipy.stats** (skewness, kurtosis) : <https://docs.scipy.org/doc/scipy/reference/stats.html>
- **sklearn — ColumnTransformer** : <https://scikit-learn.org/stable/modules/compose.html>

## Configuration recommandée pour les graphiques

```
# Mettre en tête de votre notebook — style cohérent
import matplotlib
import seaborn as sns

sns.set_style('whitegrid')
sns.set_context('notebook', font_scale=1.1)
matplotlib.rcParams.update({
    'figure.dpi': 110,
```



```
'figure.figsize': (12, 6),
'axes.titlesize': 13,
'axes.titleweight': 'bold',
'axes.labelsize': 11,
'xtick.labelsize': 9,
'ytick.labelsize': 9,
'legend.fontsize': 9,
})

# Palette cohérente tout au long du notebook
COLORS = {
    'existing': '#1A7A6E',    # Teal
    'attrited': '#9C3D38',   # Rose
    'accent': '#B07D3A',     # Or
    'neutral': '#5B3D8F',    # Violet
}
```